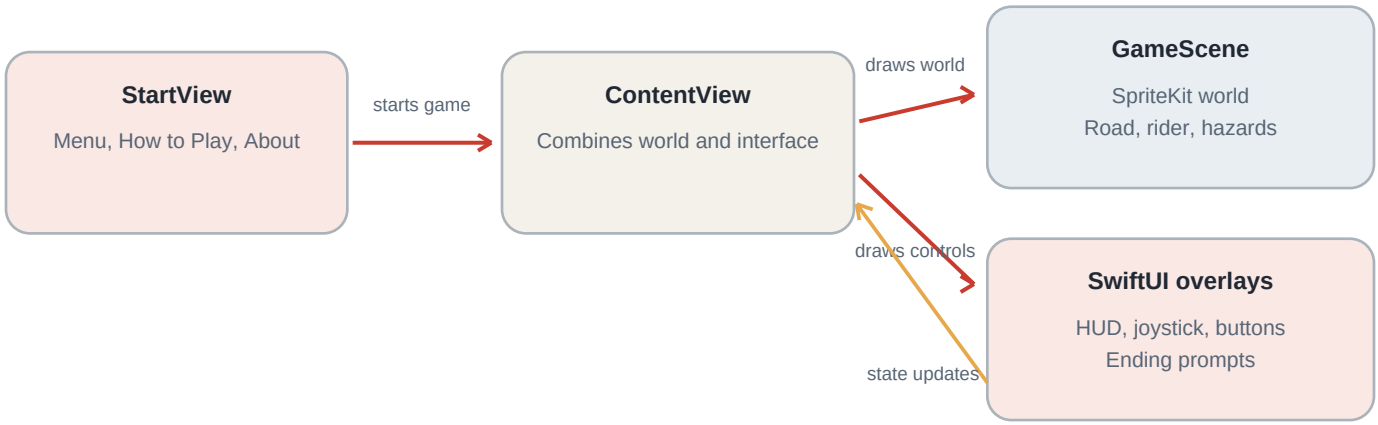
How Deliver While Hot Works

A friendly map of the SwiftUI interface, SpriteKit game world, gameplay rules, and tests.

The big picture

The app uses two Apple drawing tools. **SwiftUI** builds menus, buttons, and the heads-up display. **SpriteKit** draws the road and moving characters. They work together rather than replacing one another.



The most useful mental model

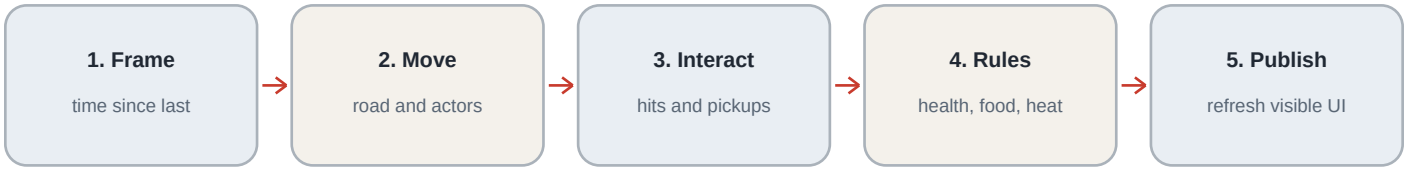
ContentView is the adapter. It puts a SpriteKit scene behind SwiftUI controls. When the player acts, SwiftUI calls a method on GameScene. When the game state changes, GameScene reports it back so the hearts, food, temperature, and ending screen can redraw.

A few words in plain language

Term	Meaning here
View	A piece of SwiftUI interface, such as the menu or an action button.
Scene	The SpriteKit stage. GameScene owns the scrolling world and its events.
Node	An object in a SpriteKit scene: a sprite, a warning marker, or a projectile.
State	The values that describe the current run, such as health and food quality.

What happens while a run is playing

SpriteKit repeatedly asks the scene to update. The game uses the time since the previous frame so movement is based on elapsed time, not on how fast a phone happens to draw frames.



Then the next display frame begins the cycle again.

In the code

GameScene.update(_:) is the traffic controller. It advances the scrolling street, moves the player and world objects, asks hazards and encounters to update, checks interactions, and publishes changed state. Keeping this sequence in mind makes the scene less mysterious.

Objects have different jobs

Code	Job	Example
GameScene	Coordinates the run and the order of events.	Scrolls the road and checks contacts.
PlayerNode	Moves and animates the rider and scooter.	Receives steering and block input.
HazardNode / EncounterNode	Represent ordinary hazards and special human or bird encounters.	A cat pounce or a thief warning.
ProjectileNode / BonusNode	Move through the world and resolve their own interactions.	A knife or a roadside food bonus.

Layer order is like paper cutouts

The road is drawn underneath its markings. Sidewalks and frontage sit beside it; props sit over the ground. Hazards, projectiles, and rider sit above scenery. Weather effects sit above the world art, while the SwiftUI HUD and controls are over the SpriteKit view. **zPosition** is SpriteKit's front-to-back ordering value.

A texture is the picture; the node is the picture's place in the game.

How the rules stay understandable

GameState stores the current numbers. GameRules contains calculations that can be checked without drawing a screen or starting SpriteKit. Separating these ideas is one of the most helpful maintainability choices in this prototype.

Example: eating

```
Tap Eat -> GameScene asks GameRules.consumeFood(state)
           -> one heart is restored, 0.25 food is spent
           -> ContentView receives the new state
           -> the HUD redraws
```

Some current rules

Value	Current rule
Health	Starts at 3 hearts. Obstacle damage removes 1 and health cannot go below zero.
Food	Starts full at 1.0, represented as four slices. Eating costs 0.25 and restores one heart.
Block	A blocked hit costs 0.25 food. A perfect parry is checked within 0.12 seconds of starting the block.
Recovery bonus	The roadside delivery bonus adds 0.50 food, capped at full.
Temperature	Falls by 0.027 each second and stops at zero.
Delivery	Requires health above zero, temperature at least 0.20, and food at least 0.25.

What the tests are for

A unit test calls a small rule directly with a known starting value, then checks the answer. For example: start at 0.50 food, block one hit, and expect 0.25. This is fast and isolates the arithmetic from the visuals.

The UI test checks a user path: open How to Play, open About, start a game, and find the gameplay controls. It does not prove that the steering feels right or that every hazard animation looks good. Those still need a human play-test.

> Tests catch regressions; they do not replace playing the game.

A practical map for changing things

If you want to change...	Start by looking at...
Menu or instructions	StartView.swift and Localizable.xcstrings
HUD or touch controls	ContentView.swift and UI/GameControlsView.swift
Game values and balancing	Game/GameState.swift, Game/GameRules.swift, and Game/GameConstants.swift
Scrolling world or collision	Game/GameScene.swift
A hazard or projectile	Game/HazardNode.swift, Game/EncounterNode.swift, and Game/ProjectileNode.swift
A visible change	Run the app in the simulator and compare screenshots or short recordings.

Build and test

```
xcodebuild test \
  -project DeliverWhileHot.xcodeproj \
  -scheme DeliverWhileHot \
  -destination 'platform=iOS Simulator,name=iPhone 16' \
  CODE_SIGNING_ALLOWED=NO
```

The GitHub Actions workflow runs the same test scheme on a hosted macOS machine. A green check means that the build and the automated tests passed there; it does not certify art quality or balance.

Current review notes

The supplied gameplay recording shows a thief throw pose that looks much larger than the entrance pose. The current code holds its SpriteKit sprite at a fixed size and uses fixed-size animation textures, so the remaining mismatch appears to be the visible artwork filling the frame differently. Compare the actual frame silhouettes at device scale before changing the physics body or collision behavior.

The recording also has occasional uneven motion. A recording alone cannot say whether that is a frame-time hitch, simulator load, or a movement-tuning issue. Use Xcode's Metal/System Trace or Instruments on a real device before making a performance claim.

Before sharing publicly

- Confirm redistribution and attribution terms for every art and audio file; AUDIO_LICENSES.md still marks several sources as unverified.
- The repository tracks personal Xcode workspace files. .gitignore prevents new local state, but already tracked files must be untracked once before a public push.
- The current machine could not finish an Xcode test build because its storyboard compiler reports the iOS platform as not installed and CoreSimulator reports no available runtimes.

Remote crash reports can help once external testers use signed builds. Firebase Crashlytics has a no-cost plan, but it requires a Firebase project, an app configuration file, an SDK, and a privacy decision. For this prototype, CI plus local simulator testing is a sensible first step; add telemetry only when you are ready to collect crash data.

Remote monitoring references: Firebase pricing and Crashlytics setup for Apple platforms.

> Keep the feedback loop small: change one thing, build, test, then play that exact change.